

Converting a Laravel Internal Messaging System into a Cloud SaaS Application

Student Name: _____

Student ID: _____

Course: _____

Instructor: _____

Project: Laravel Multi-Tenant SaaS Messaging System

Date: May 13, 2026

This report explains, step by step, how a normal Laravel internal messaging project was changed into a basic SaaS application where multiple companies can subscribe, use the same application, and keep their data separated.

1. Original Project

The starting point was a simple Laravel application for internal messaging. It had users, messages, a message form, an inbox, sent messages, and the ability to mark a message as read.

- Framework: Laravel 13.
- Database: SQLite for local development.
- Main feature: internal messages between users.
- Main files before SaaS conversion: Message model, MessageController, messages migration, and one Blade view.

2. SaaS Goal

SaaS means Software as a Service. Instead of installing a separate project for every company, one Laravel application is hosted and many companies use it through subscriptions.

- Each company is a tenant.
- Each tenant has its own users.
- Each tenant has its own messages.
- Each tenant has its own subscription.
- A Super Admin can manage companies, subscription state, and enabled modules.

3. Research: Multi-Tenant and Multi-Modules Packages

Laravel supports this kind of architecture through community packages. In this assignment, the packages were researched and the same concepts were implemented manually in a simple way.

Concept	Package	Purpose
Multi-tenant	stancl/tenancy	Automatic multi-tenancy for Laravel, with Laravel 13 support.
Multi-tenant	spatie/laravel-multitenancy	Makes Laravel tenant-aware and can work with one or multiple databases.
Multi-modules	nwidart/laravel-modules	Organizes features into separate modules with routes, models, views, migrations

References: <https://packagist.org/packages/stancl/tenancy>, <https://spatie.be/docs/laravel-multitenancy>, <https://laravelmodules.com/docs/13/getting-started/introduction>

4. Chosen Implementation Approach

For this assignment, the SaaS idea was implemented using single-database multi-tenancy. This means all companies share the same database, but every company-owned record stores a `company_id`. The application always filters records using `company_id`.

```
Message::where('company_id', $currentCompanyId)->get();
User::where('company_id', $currentCompanyId)->get();
```

This is a simple and clear way to demonstrate the SaaS concept without requiring separate databases, subdomains, or payment gateways.

5. Database Changes

The database was changed from a normal messaging schema into a SaaS schema.

- companies table: stores each tenant company.
- subscriptions table: stores the plan, status, seats, price, start date, and end date for each company.
- modules table: stores available application modules.
- company_module table: stores which modules are enabled for each company.
- users.company_id: connects users to a company.
- users.role: stores employee, company_admin, or super_admin.
- messages.company_id: connects every message to one company.

```
companies
subscriptions
modules
company_module
users.company_id
messages.company_id
```

6. Model Relationships

New Eloquent models and relationships were added to represent SaaS ownership.

- Company has one Subscription.
- Company has many Users.
- Company has many Messages.
- Company belongs to many Modules.
- User belongs to Company.
- Message belongs to Company, Sender, and Recipient.

7. Controller Logic

The MessageController was changed so the page works inside the selected company tenant.

- The selected company is read from company_id in the query string.
- Only users from the selected company are loaded.
- Only inbox and sent messages from the selected company are loaded.
- A message can only be sent if sender and recipient belong to the same company.
- A message can only be sent if the company is active or trialing.
- A message can only be sent if the subscription is active or trialing.
- A message can only be sent if the Messaging module is enabled.

```
abort_unless($message->company_id === $user->company_id && $message->recipient_id === $userId, 403);
```

8. Super Admin Control

A Super Admin console was added to the page. It can manage company status and enabled modules.

- The Super Admin can see all companies.
- The Super Admin can see user and message counts per company.
- The Super Admin can change company status: active, trialing, or suspended.
- The Super Admin can enable or disable modules per company.

9. Multi-Modules Implementation

The project demonstrates modules through a modules table and a company_module pivot table. Each company can have different modules enabled. For example, one company can have Messaging, Subscriptions, Reports, and Company Management, while another company can have only Messaging and Subscriptions.

In a larger production project, the package `nwidart/laravel-modules` could be installed to physically separate module files into folders such as `Modules/Messaging`, `Modules/Subscriptions`, and `Modules/Admin`.

10. Verification

The project was verified after the SaaS conversion.

- Database migrations ran successfully.
- Demo companies, subscriptions, modules, users, and messages were seeded.
- The browser showed the SaaS dashboard with Super Admin console and tenant workspace.
- Automated tests passed: 5 tests, 5 passed.
- A test confirms users cannot send messages across different companies.

11. Screenshot

The following screenshot shows the converted SaaS dashboard running locally.

The screenshot displays a web interface for a 'Cloud SaaS prototype' titled 'Multi-Tenant Messaging System'. It features two main sections. The top section, titled 'Tenant company', contains two dropdown menus: the first is set to 'Blue Orbit Studio' and the second is empty. The bottom section, titled 'Super Admin Console', shows the tenant name 'Blue Orbit Studio', the email 'admin@blueorbit.test', and a status '0 users · 0 messages · Starter'. A red 'suspended' badge is visible on the right side of this section.

Cloud SaaS prototype

Multi-Tenant Messaging System

Tenant company

Blue Orbit Studio

Company user

Super Admin Console

Blue Orbit Studio

admin@blueorbit.test

0 users · 0 messages · Starter

suspended

12. Conclusion

The original Laravel project was a normal internal messaging system. It was converted into a basic Cloud SaaS application by adding companies as tenants, separating users and messages with `company_id`, adding subscriptions, adding modules, and adding a Super Admin interface to control companies. This fulfills the assignment concept of multi-tenant and multi-module SaaS in Laravel.

Project Path

C:\Users\JoyBoy\Documents\Codex\2026-05-13\github-ai\internal-messaging